

Face Detection Features

Structured Notes

LBP, HOG, and Haar Feature

Descriptors
for Face Detection and Recognition

Contents

1	Introduction to On-Edge Face Detection	2
1.1	Why Feature Descriptors for the Face Region	2
2	Local Binary Pattern (LBP)	2
2.1	Pixel Values and Image Bits	2
2.2	LBP Algorithm: 3×3 Region Threshold	3
2.2.1	Worked Example 1: Center Pixel = 4	3
2.3	Binary-to-Decimal Conversion	3
2.3.1	Worked Example 2: Center Pixel = 100	3
2.4	Sliding the Window Across the Image	4
2.5	LBP Histogram (Feature Vector)	4
3	Histogram of Oriented Gradients (HOG)	4
3.1	Stage 1: Preprocessing — Resize to 1:2 Ratio	5
3.2	Stage 2: Gradient Computation	5
3.2.1	Handling Border Pixels	5
3.3	Stage 3: Magnitude and Direction	5
3.3.1	Worked Numerical Example	6
3.4	Stage 4: The Four Methods of Histogram Computation	6
3.4.1	Method 1 — 180-Dimensional Histogram	6
3.4.2	Method 2 — 9 Bins of 20 Degrees, Angle Only	6
3.4.3	Method 3 — 9 Bins, Sum of Magnitudes	6
3.4.4	Method 4 — Proportionate Assignment	6
3.5	Stage 5: Cells and Block Normalization	7
3.6	Stage 6: Total Feature Vector Length	7
3.7	Grayscale vs RGB Computation	7
4	Haar Features	7
4.1	Four Haar Feature Types	8
4.2	History of Haar Features	8
4.3	Stages of the Haar Algorithm	8
4.3.1	Stage 1: Grayscale Conversion	8
4.3.2	Stage 2: Integral Image Computation	8
5	End-to-End Face Detection Pipeline	9
6	Summary	9

1 Introduction to On-Edge Face Detection

On-Edge Processing

On-edge processing refers to the computation carried out directly inside a hardware device (for example, a CCTV camera or a mobile phone) rather than sending the data to a server. The hardware holds a small processor that is responsible for taking small decisions instantly, such as face detection or face identification.

A traditional CCTV setup captures video and transmits it through a Network Video Recorder (NVR) to storage, and different software applied on that storage performs face detection, intrusion detection, person tracking, or identification of missing objects. However, many of these tasks do not need a server-based configuration. The CCTV file captures the videos, and a very small processor inside the CCTV is responsible for taking many small decisions. One of the decisions that can be taken on the device is face detection or face identification.

Such on-edge applications are already available in mobile cameras where the system detects the face region and reports the age of the person, and also where the system detects whether a person is smiling or not.

1.1 Why Feature Descriptors for the Face Region

The face region of a human body contains a component called **textures**. A texture is a pattern (or a certain type of pattern) that repeats several times. If a human face is observed at the micro level, certain types of patterns repeat again and again. The aim of the algorithms discussed here is to identify these patterns, and these patterns are mainly responsible for taking a decision on how to detect the face region in a camera.

Three Feature Algorithms Covered

Three feature descriptor algorithms are covered for face detection:

- **LBP** — Local Binary Pattern
- **HOG** — Histogram of Oriented Gradients
- **Haar** — Haar Features

Out of these three, Haar is a very popular architecture and is used by many industries, but HOG and LBP also have popularity.

2 Local Binary Pattern (LBP)

Local Binary Pattern

LBP stands for Local Binary Pattern. It is one of the effective feature descriptors and is mainly responsible for texture description. LBP considers a small rectangular region in the image and identifies or thresholds the neighboring pixels based on the center pixel.

2.1 Pixel Values and Image Bits

When any image is considered, the image is represented by a number of pixels. A pixel is one small element in an image and is the unique composition of many such pixels gives rise to one image. The image has pixel values (also called gray values), and for an 8-bit image these pixel values vary from 0 to 255, that is, there are 256 gray values ($2^8 = 256$).

If the image uses multiple bits, the number of pixel values will depend on that. For 9 bits, $2^9 = 512$ gray values are possible (0 to 511). Most of the pictures and cameras in daily life (mobile, camcorder,

pixel-based cameras, DSLR cameras) use 8-bit images. Remote sensing satellite images go up to 16 bits, which gives more bits to represent values.

Trade-off

If the number of bits of an image is increased, it will increase the amount of storage required and the computation required for calculating large numbers. A particular number of bits is chosen so that computation power is less while the resolution is still enough to display the image properly.

2.2 LBP Algorithm: 3×3 Region Threshold

LBP considers a 3×3 region in an image. At a particular pixel location, a 3×3 region is chosen. The center pixel of this region is compared with each of its neighboring pixels:

Thresholding rule:

$$b_i = \begin{cases} 1 & \text{if } P_{\text{center}} \geq P_{\text{neighbor}}^{(i)} \\ 0 & \text{if } P_{\text{center}} < P_{\text{neighbor}}^{(i)} \end{cases}$$

If the center pixel is greater than the neighboring pixel, a 1 is placed at that neighbor position; otherwise a 0 is placed.

2.2.1 Worked Example 1: Center Pixel = 4

Consider a 3×3 window with center pixel equal to 4 and neighbors 1, 8, 5, 5, 3, 7, 2, 1. Compare the center with each neighbor:

- 4 vs 1: $4 > 1 \Rightarrow 1$
- 4 vs 8: $4 < 8 \Rightarrow 0$
- 4 vs 5: $4 < 5 \Rightarrow 0$
- 4 vs 5: $4 < 5 \Rightarrow 0$
- 4 vs 3: $4 > 3 \Rightarrow 1$
- 4 vs 7: $4 < 7 \Rightarrow 0$
- 4 vs 2: $4 > 2 \Rightarrow 1$
- 4 vs 1: $4 > 1 \Rightarrow 1$

Larger neighbor pixels (8, 5, 5, 7) receive a 0 and smaller neighbors (3, 1, 1, 2) receive a 1.

2.3 Binary-to-Decimal Conversion

Once the 8 binary digits are obtained, they are converted into a decimal number. Because there are 8 neighbors, the result is an 8-bit number whose maximum value is 255.

Clockwise Convention

The sequence must start from a fixed starting pixel and proceed in the clockwise direction. Different conventions are possible: one can treat the first pixel as bit 0 or as the highest-order bit. Typically the opposite direction convention is used so that bits are labeled 0, 1, 2, ..., 7 across the 8 neighbors.

For the first example, the binary pattern read clockwise gives bits 1, 1, 1, 0, 1, 0, 0, 0, and the decimal conversion yields:

$$2^4 + 2^2 + 2^1 + 2^0 = 16 + 4 + 2 + 1 = 23$$

This value 23 is the LBP pattern assigned to that center pixel.

2.3.1 Worked Example 2: Center Pixel = 100

Consider a center pixel of value 100 with neighbors 107, 93, 99, 78, 108, 113, 104, 101.

- 100 vs 107: $107 > 100 \Rightarrow 0$
- 100 vs 93: $93 < 100 \Rightarrow 1$
- 100 vs 99: $99 < 100 \Rightarrow 1$
- 100 vs 78: $78 < 100 \Rightarrow 1$
- 100 vs 108: $108 > 100 \Rightarrow 0$
- 100 vs 113: $113 > 100 \Rightarrow 0$
- 100 vs 104: $104 > 100 \Rightarrow 0$
- 100 vs 101: $101 > 100 \Rightarrow 0$

Read clockwise, the bit sequence is 0, 1, 1, 1, 0, 0, 0, 0. Using the convention that leftmost read is the lowest bit, the decimal value is:

$$2^7 + 2^6 + 2^5 = 128 + 64 + 32 = 24$$

(using the clockwise convention described during computation). This is the LBP value assigned to that center pixel.

2.4 Sliding the Window Across the Image

LBP is applied at every pixel location of the image. A center-sliding 3×3 window moves across the entire image exactly the same way a convolution operation slides a kernel: at each position, the LBP value is computed for the center pixel. The result is an output image whose pixel values are the LBP codes.

When applied to a human face, the LBP output highlights the texture quality or texture regions present in the face area. The output accumulates the binary strings available in the image, which is how LBP extracts meaningful features from the face region of a human face (or human object).

2.5 LBP Histogram (Feature Vector)

After obtaining the LBP value at every pixel, the next step is to plot its histogram. A histogram calculates the number of occurrences of each value.

LBP histogram:

- x -axis: LBP values (from 0 to 255 for 8-bit)
- y -axis: frequency of occurrence of each LBP value

The histogram records how many times each LBP code appears in the image.

For example, value 23 may occur many times, value 7 twice, value 5 twice, and so on. The plot of these frequencies is called the **LBP histogram** or **LBP feature vector**.

Why This Matters

The LBP histogram is one of the important features used for feature extraction, most commonly for computing the face area in images.

3 Histogram of Oriented Gradients (HOG)

HOG

HOG stands for Histogram of Oriented Gradients. It is a famous feature used for feature description in computer vision. This algorithm was initially developed for object detection. HOG is particularly effective when the goal is to find out the shapes and outlines of objects in images.

For a face area, important characteristics include the shape of the person, nose shape, eye shape, and

lip shape. The outlines of any type of structure in the face area must also be found. HOG identifies these shape-and-outline descriptions, which is the best way to represent the face of a human being.

HOG is structured in six to seven stages.

3.1 Stage 1: Preprocessing — Resize to 1:2 Ratio

In the preprocessing stage, the original input image is resized so that the number of rows and columns is in a 1 : 2 ratio.

Resize Example

If the original image is 100×130 , it must be resized to something like 100×200 or 50×100 , so that the row-to-column ratio is 1 : 2.

3.2 Stage 2: Gradient Computation

The gradient of the image is computed. Gradient computation is done on a 3×3 block of pixels. The image is treated as a 2D entity with a row index and a column index; equivalently, x and y coordinates, where the horizontal direction is y and the vertical direction is x .

For a center pixel at (x, y) , the eight surrounding pixels are:

$$(x-1, y-1), (x-1, y), (x-1, y+1), (x, y-1), (x, y+1), (x+1, y-1), (x+1, y), (x+1, y+1)$$

Because the image is two-dimensional, there are two gradient components: a gradient in the x direction (G_x) and a gradient in the y direction (G_y).

Gradient formulas:

$$G_x = I(x, y+1) - I(x, y-1)$$

$$G_y = I(x-1, y) - I(x+1, y)$$

G_x is the difference between the right and left neighbors of the center, and G_y is the difference between the top and bottom neighbors.

3.2.1 Handling Border Pixels

A border pixel does not have all eight neighbors, so the gradient cannot be computed directly at the edges. Two options are available:

- **Omit border pixels:** leave out one row and one column on each side.
- **Zero padding:** append one extra row and one extra column on top, bottom, left, and right, with all values set to zero. This is the same zero-padding process used in CNN algorithms.

Zero padding allows the algorithm to compute gradients for the border pixels as well, although the computation is a kind of pseudo calculation because all the appended values are zero.

3.3 Stage 3: Magnitude and Direction

From G_x and G_y , the magnitude and direction of the gradient are computed.

Gradient magnitude and direction:

$$|G| = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

Because G_x^2 and G_y^2 are already non-negative, taking an absolute value is not strictly required. The magnitude represents the **strength of the edge** or the strength of the variation of pixel values in the

image. The direction (the $\tan^{-1}$ value) tells **on which direction there is a change** in pixel value. The direction is to be calculated for every grid cell.

3.3.1 Worked Numerical Example

Consider a center pixel with value 85 and surrounding values such that $I(x, y + 1) = 89$, $I(x, y - 1) = 78$, $I(x - 1, y) = 68$, $I(x + 1, y) = 56$. Then:

$$G_x = 89 - 78 = 11$$

$$G_y = 68 - 56 = 12$$

$$|G| = \sqrt{11^2 + 12^2} = \sqrt{121 + 144} = \sqrt{265} \approx 16.27$$

$$\theta = \tan^{-1}\left(\frac{12}{11}\right)$$

The same calculation is done at every pixel location, producing a magnitude image and a direction ($\theta_{x,y}$) image for the whole input image.

3.4 Stage 4: The Four Methods of Histogram Computation

HOG provides four methods (Method 1 through Method 4) for building the histogram from the gradient data.

3.4.1 Method 1 — 180-Dimensional Histogram

In Method 1, the gradient direction $\theta_{x,y}$ (in the range 0° to 180°) is used directly. This produces a 180-dimensional feature vector because each integer angle value from 0 to 179 gets its own histogram bin. This vector may have many zero components but its ultimate dimension is 180.

3.4.2 Method 2 — 9 Bins of 20 Degrees, Angle Only

In Method 2, the 0° to 180° range is divided into 9 bins of 20 degrees each:

$$[0^\circ, 20^\circ), [20^\circ, 40^\circ), [40^\circ, 60^\circ), \dots, [160^\circ, 180^\circ]$$

For each bin, the count of pixels whose angle falls into that bin is recorded. The result is a 9-dimensional vector. Method 2 uses the angle **only** (not the magnitude).

Why Method 2 Helps

Previously Method 1 required a 180-dimensional feature vector for representation. Method 2 uses only 9 dimensions, which is much easier for computation. This is the main advantage: the sparse 180-D vector is converted into a dense 9-D vector.

3.4.3 Method 3 — 9 Bins, Sum of Magnitudes

Method 3 is like Method 2 but instead of counting the number of pixels in each angular bin, it *adds up the gradient magnitudes* of the pixels whose angles fall into that bin. Both the gradient magnitude image and the gradient direction image are created from the input. For each of the nine angular bins, the magnitude values of all matching pixels are summed and placed into that bin.

3.4.4 Method 4 — Proportionate Assignment

Method 4 refines Method 3 by assigning the magnitude *proportionately* across the two nearest bins, rather than fully to a single bin.

For example, suppose the magnitude is 13.6 and the direction is 36 degrees. The direction 36 falls in the $[20^\circ, 40^\circ)$ bin and would be assigned entirely to that bin in Method 3. However, 36 is closer to 40 than to 20, so in Method 4 the value is distributed across the $[20, 40)$ and $[40, 60)$ bins in proportion to their distances:

Proportionate split:

$$\begin{aligned}\text{share to } [20, 40) &= 13.6 \times \frac{40 - 36}{20} = 13.6 \times \frac{4}{20} \\ \text{share to } [40, 60) &= 13.6 \times \frac{36 - 20}{20} = 13.6 \times \frac{16}{20}\end{aligned}$$

In this way, rather than assigning the gradient value directly to one bin, the magnitude is distributed with some proportion to the two neighboring bins.

3.5 Stage 5: Cells and Block Normalization

The input image is divided into small cells. Each cell is 8×8 pixels, giving a total of 64 pixels per cell. For every 8×8 cell, Method 2, 3, or 4 gives a 9-dimensional feature vector (Method 1 gives 180-dimensional).

Next, four such 8×8 cells are grouped to form a 16×16 block for normalization. Each 16×16 block contains four cells, producing a concatenated vector of $4 \times 9 = 36$ values.

Block normalization:

Label the 36 values within a block as $a_1, a_2, \dots, a_{36}$. Compute

$$x = \sqrt{a_1^2 + a_2^2 + \dots + a_{36}^2}$$

Then normalize each value:

$$a_i \leftarrow \frac{a_i}{x}, \quad i = 1, 2, \dots, 36$$

3.6 Stage 6: Total Feature Vector Length

Because the input image is resized to a 1 : 2 row-to-column ratio, the number of 16×16 blocks in each dimension matches that ratio. If there are 7 blocks in the row direction, there will be 14 blocks in the column direction. This gives $7 \times 14 = 98$ total blocks.

Total HOG feature vector length:

$$\text{Blocks} \times \text{Features per block} = 98 \times 36 = 3528$$

The 3528-dimensional vector is the final HOG feature descriptor of the image.

3.7 Grayscale vs RGB Computation

HOG can be computed in two ways:

- Convert the image to grayscale and compute, yielding one channel only.
- Compute HOG separately on the red, green, and blue channels and combine.

Both methods are acceptable.

4 Haar Features**Haar Features**

The **Haar** algorithm is a very popular, fast, and efficient method used primarily for detecting

faces. It identifies the face region in any kind of image by computing the difference between a fixed-size window and its nearest neighboring window in the image.

Haar considers a small fixed-size window and tries to differentiate between that window and its neighboring window. In a face, consecutive windows placed on the face region produce a large difference in gray pixel values, whereas consecutive windows in the hair region or the background produce little difference. Haar tests multiple window positions to find candidate face locations based on these differences.

4.1 Four Haar Feature Types

Haar considers four different types of features for detecting the face region:

Feature Type	How It Is Computed
Edge feature	Two vertically-aligned or horizontally-aligned windows. Take the sum of all pixels in the top region and the sum in the bottom region, and take their difference. The same applies for left vs right.
Line feature	Three consecutive windows. Compute (sum of first) + (sum of third) – (sum of the middle), equivalent to white pixel combination minus black pixel. Can be oriented vertically or horizontally.
Rectangle feature	Four windows arranged as a 2×2 rectangle. Sum the two diagonal blocks and subtract the other two (all white pixels subtracted from all black pixels).
Correlation feature	Based on the correlation between regions, completing the set of four feature types considered.

Essentially, Haar features consider the difference between various regions in the image.

4.2 History of Haar Features

The Haar feature was invented in 1909 by a scientist named Haar, which is why it is called the Haar feature. The original work was proposed for some other kind of functional representation. The feature was not used for many years. In the 2000s, this algorithm gained popularity for face recognition applications, and that is why it is now called the Haar feature for face recognition.

4.3 Stages of the Haar Algorithm

4.3.1 Stage 1: Grayscale Conversion

The original image is first converted into a grayscale image. A simple conversion is:

$$\text{gray} = \frac{R + G + B}{3}$$

4.3.2 Stage 2: Integral Image Computation

After grayscale conversion, the integral image (I_2) is computed. The integral image is a new image whose pixel value at a location is the sum of the pixels in a small region area up to that location.

Worked Example

Consider a small region containing the pixel values 5, 2, 1, 4. The integral image value at that location is:

$$I_2 = 5 + 2 + 1 + 4 = 12$$

Once this is computed across the image, the integral image holds cumulative sums that can then be used for efficient Haar feature evaluation at any scanning window position.

5 End-to-End Face Detection Pipeline

All the algorithms described above share a common structure: given a captured input image, the first task is to identify the face region, and on that region any classifier can be applied to identify whose face it is. In short, the algorithms concentrate on two things:

1. Identify where the face region is in the input image.
2. Apply a classifier on that cropped region to identify the person.

The feature descriptors (LBP, HOG, Haar) must produce very different values for the face region compared to the dress area or the background. The face has a lot of variation, so it is expected that due to this variation a higher or completely different value is obtained for the face region. The descriptor response identifies the cropped region of the face on which detection focuses.

In deep learning, this cropped region is then fed as input to any CNN algorithm to extract a set of features, and a classifier or search-based approach matches the features against a database of different faces to output the identity of that person.

6 Summary

- **On-edge processing** allows face detection to run directly inside small processors in CCTVs and mobile cameras without a server.
- **LBP** thresholds eight neighbors of a center pixel in a 3×3 window, forms an 8-bit binary code in clockwise order, and converts it to a decimal value in $[0, 255]$.
- The **LBP histogram** plots the frequency of each LBP code across the image and forms the LBP feature vector.
- **HOG** resizes the image to $1 : 2$, computes gradients G_x and G_y , the magnitude $\sqrt{G_x^2 + G_y^2}$, and the direction $\tan^{-1}(G_y/G_x)$.
- HOG has four histogram computation methods: Method 1 uses the raw 180-D angle histogram; Methods 2–4 use 9 bins of 20 degrees each, with Method 3 summing magnitudes and Method 4 distributing magnitudes proportionately across bins.
- HOG uses 8×8 cells grouped into 16×16 blocks (36 values per block), normalized by the L2 norm of the block. A $1 : 2$ image with $7 \times 14 = 98$ blocks produces a $98 \times 36 = 3528$ -dimensional feature vector.
- **Haar features** capture regional pixel differences using edge, line, rectangle, and correlation features, supported by the integral image for fast evaluation.
- All three descriptors produce distinctively different values over the face region, enabling detection of the face before classification.